

DELIVERYBENCH-RL: Reinforcement Learning for Long-Horizon, Multimodal Embodied Agents under Realistic Constraints

[TODO: author list]
[TODO: affiliation]

Abstract

Coding agents have shown that strong autonomy, tool use, and proactive reasoning can be elicited through long-horizon harnesses and agentic training. We ask whether the same agentic capacity can be granted to a *real-world, embodied* setting with multimodal input. DELIVERYBENCH introduced a long-horizon embodied delivery environment with realistic constraints—time, energy, and safety—and revealed two core deficiencies that persist even in frontier vision–language models (VLMs): a lack of long-horizon planning, and a failure to *proactively* satisfy task constraints, including resource limits and *implicit* commonsense rules (e.g. safety and traffic rules) that the same models recall reliably in isolated question answering. We present DELIVERYBENCH-RL, which turns this benchmark into a trainable environment for multimodal agentic reinforcement learning (RL) by adding step-wise reward shaping, an oracle planner, a multi-dimensional curriculum, tool calling, and adjustable action granularity. We train and compare RL baselines (PPO, GRPO, SUPO) against prompt-scaffolded frontier VLMs, and dissect the extent to which RL grants proactive mastery of individual implicit constraints. [TODO: 1–2 sentence headline result.]

1 Introduction

Recent progress in *coding agents* has shown that strong autonomy, fluent tool use, and proactive, self-directed reasoning can be *elicited and trained* rather than merely prompted, given a long-horizon harness and an agentic training loop. A natural question follows: can the same agentic capacity be granted in a *real-world embodied* environment with *multimodal* input, where an agent must perceive, plan over long horizons, and respect physical and social constraints?

DELIVERYBENCH was, to our knowledge, the first long-horizon agentic environment for real-world embodied tasks to couple this multimodal, egocentric setting with *realistic constraints*—a time budget (a fixed shift), energy and battery limits, and safety/social rules such as obstacles and traffic signals. Studying frontier VLMs in DELIVERYBENCH exposed two core deficiencies that survive even at the frontier:

1. **Weak long-horizon planning.** Models fail to compose the many steps of a delivery (select, route, pick up, deliver, replan) into a plan that respects a global time/energy budget.
2. **Non-proactive constraint handling.** Models do not *proactively* use task constraints—both resource limits and *implicit* commonsense rules—to guide their reasoning. Crucially, these rules (e.g. “do not cross on red”, “go around an obstacle”) are recalled reliably in *direct question answering*, yet when the same rule must be applied from within a complex, long-horizon task state, the model fails to invoke it. This gap between *knowing* a rule and *acting* on it is a latent robustness and safety problem.

DELIVERYBENCH-RL builds directly on this foundation and makes *multimodal agentic RL training* in the environment feasible. We integrate (i) step-wise **reward shaping** with a customizable, environment-aware interface; (ii) an **oracle planner** that yields globally optimal trajectories; (iii) a multi-dimensional **curriculum**; (iv) **tool calling** for adaptive context loading; and (v) adjustable **action granularity**. The oracle planner in particular gives us infrastructure for imitation learning, reward normalization, automatic QA/unit-test construction, and fine-grained failure analysis, while action granularity lets the same environment probe high-level planning *and* low-level visual navigation (spatial reasoning, safety, social navigation).

Contributions.

1. We extend DELIVERYBENCH into DELIVERYBENCH-RL, an RL-ready, long-horizon, multimodal embodied environment with an oracle planner, multi-dimensional curriculum, step-wise (and customizable) reward shaping, tool calling, and adjustable action granularity (Section 3).
2. We conduct, to our knowledge, the first study of multimodal agentic RL in a real-world long-horizon embodied environment, training and comparing baseline algorithms (PPO, GRPO, SUPO) and prompt-scaffolded frontier VLMs (Section 4.1).
3. We analyze how well RL-trained models acquire *proactive* handling of distinct implicit constraints, and isolate the *unit* capabilities models still struggle to master (Section 4.3).

2 Related Work

[**TODO:** Long-horizon / agentic RL environments; embodied & navigation benchmarks (ALFWorld, Habitat, etc.); LLM/VLM agents and tool use; agentic RL (PPO/GRPO and variants); curriculum learning; imitation from oracle/expert. Position DELIVERYBENCH-RL against Table 1.]

3 DELIVERYBENCH-RL: An RL-Ready Embodied Agentic Environment

DELIVERYBENCH-RL starts from DELIVERYBENCH’s simulated city-delivery task—a courier completes food deliveries within a two-hour shift under time, energy/battery, and safety constraints—and adds the machinery required to *train* agents, not just evaluate them. Table 1 contrasts DELIVERYBENCH-RL with DELIVERYBENCH and with other long-horizon agentic environments. The five additions below are ordered by their centrality to enabling RL.

3.1 Reward Shaping: From Terminal Profit to Expressive Step-wise Signal

In DELIVERYBENCH the only reward signal was the *terminal* profit of a full trajectory—a sparse, high-variance target that is poorly suited to long-horizon RL. DELIVERYBENCH-RL decomposes this into a **step-wise** reward built from short-horizon *partial-success* events: response/format-schema correctness, constraint violations (e.g. collisions, red-light crossings, deadline misses), and sub-task completions (order accepted, picked up, dropped off). This makes the training signal far more expressive and densely attributable. Importantly, we expose an **interface for customizing reward shaping from environment state**, so downstream work can define new shaping terms (or safety penalties) without touching the core loop. [**TODO:** State the default reward decomposition $r_t = \sum_k w_k \phi_k(s_t, a_t)$ and list the terms ϕ_k in a table; note which are on by default.]

Table 1: DELIVERYBENCH-RL versus DELIVERYBENCH and other long-horizon agentic environments. ✓ full support, ∼ partial, ✗ none. The top block is the direct DELIVERYBENCH lineage; the bottom block (other environments) is [TODO: populate with verified entries].

	Multimodal (FPV+map)	Long horizon	Realistic constraints	Oracle planner	Curriculum	Step-wise reward shaping	Tool calling	Adjustable action granularity	RL-ready
DELIVERYBENCH-RL (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓
DELIVERYBENCH (v1)	✓	✓	✓	✗	✗	∼	✗	∼	✗
Other long-horizon agentic environments									
[TODO: Env A]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO: Env B]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO: Env C]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

3.2 Oracle Planner: Optimal Trajectories for Normalization, Imitation, and Tests

Using full global information, a search-based planner computes a (near-)optimal trajectory for any task instance. This single component powers four capabilities: (i) **reward normalization**—scoring an agent against the optimal achievable profit/cost rather than an absolute scale; (ii) **imitation learning**—oracle rollouts become expert demonstrations for behavior cloning / SFT warm-start; (iii) **automatic QA and unit tests**—each oracle decision yields a labeled “what is the correct next action / which constraint applies here” probe; and (iv) **failure analysis**—an agent trajectory can be diffed against the oracle to localize *where* and *why* it diverged. [TODO: Specify the planner (graph search over the waypoint graph; objective = maximize profit / minimize time-energy s.t. deadlines and safety); note optimality guarantees / approximations.]

3.3 Multi-Dimensional Curriculum

DELIVERYBENCH-RL warms the agent into the environment along three orthogonal axes:

- **Horizon curriculum:** progressively longer shifts and more orders per episode, growing the planning horizon.
- **Prompt curriculum:** the description of each constraint and the action space is annealed from *few-shot* examples, to explicit *instructions*, to *proactive* (the agent must surface the rule itself)—directly targeting the know-vs-act gap of Section 1.
- **Constraint curriculum:** realistic constraints and their associated actions are introduced incrementally (e.g. energy/battery, then obstacles/BYPASS, then traffic lights/WAIT).

[TODO: Table of curriculum stages × axis settings; which stage unlocks which constraint/action.]

3.4 Tool Calling for Adaptive Context Loading

Rather than dumping every observation channel into the step prompt, DELIVERYBENCH-RL exposes the four-direction first-person views, the top-down map, and *query tools* (navigation/route, order pool, energy/battery, ...) that the agent calls on demand. This **adaptive context loading** keeps each step’s context small, which is what makes long-horizon rollouts tractable to *scale*—context

Table 2: Main results. Prompt-scaffolded frontier VLMs vs. open-source RL baselines on DELIVERYBENCH-RL. Metrics: task success / #deliveries, hourly profit (profit / sim-hour), and constraint-violation rates (collisions, traffic violations, deadline misses); lower is better for violations. **[TODO: fill all cells; bold the best per column; add std over seeds]**.

Method	Type	Task performance			Constraint violations ($\downarrow$)		
		Success	#Deliv.	\$/h	Collisions	Traffic	Deadline
<i>Prompt-scaffolded frontier VLMs (no training)</i>							
[TODO: GPT-x]	zero/few-shot	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO: Claude]	zero/few-shot	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO: Gemini]	zero/few-shot	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
<i>Open-source backbone + RL (ours)</i>							
[TODO: Qwen3-VL-8B]	base (no RL)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
+ PPO	RL	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
+ GRPO	RL	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
+ SUPO	RL	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

grows with what the agent chooses to inspect, not linearly with the horizon. **[TODO: List the tools and their costs (if any); note which are query-only vs state-changing.]**

3.5 Adjustable Action Granularity

Finally, DELIVERYBENCH-RL makes action granularity a configurable axis. At a coarse granularity the environment probes high-level *long-horizon planning and constraint understanding*; at a fine granularity, navigation is driven directly from the egocentric FPV and map, letting the same environment evaluate *spatial reasoning, safety, and social navigation* (e.g. noticing a road obstacle in the front view and choosing to go around it, or waiting at a red light). This is what lets a single benchmark span planning and perception-grounded control. **[TODO: Define the granularity levels (e.g. macro NAVIGATE vs primitive MOVE(DIRECTION)) and what each evaluates.]**

4 Experiments

Setup. **[TODO: Backbone open-source VLM (e.g. Qwen3-VL-8B), frontier VLMs for the scaffolded baselines, RL framework, train/eval splits, seeds, compute, evaluation metrics (success / deliveries, hourly profit, constraint-violation rates, deadline misses), and which curriculum schedule is used.]**

4.1 Main Results: Frontier Scaffolding vs. RL Baselines

Table 2 compares prompt-scaffolded frontier VLMs against an open-source backbone trained with PPO, GRPO, and SUPO. **[TODO: 1–3 sentences of takeaway: does RL on a small open model close / exceed the gap to scaffolded frontier models, and on which metric (planning/profit vs. safety/violations)?]**

4.2 Curriculum Dynamics: Staged Emergence of Unit Tasks

Figure 1 tracks reward and per-unit-task success across training. **[TODO: Takeaway: which unit tasks emerge early vs. late; whether the curriculum is necessary for the hardest units (e.g. proactive safety) to emerge at all.]**

Figure placeholder — render from results.

Figure 1: **Reward / per-unit-task success vs. training steps**, annotated with curriculum-stage boundaries (vertical dashed lines). *What to plot:* x-axis = environment / training steps; left y-axis = mean episode reward (or hourly profit); overlaid curves = success rate of each *unit task* (accept, navigate, pickup, dropoff, energy-management, obstacle-avoidance, traffic-compliance). The figure should show each unit capability *emerging* in turn as the corresponding curriculum stage unlocks, rather than all at once. Optionally add an ablation curve (no curriculum) to show slower / non-emergent learning.

4.3 Unit Tests: Proactive Mastery of Implicit Constraints

Figure 2 isolates, per constraint, the gap between *knowing* a rule (direct QA) and *proactively acting* on it inside a long-horizon task. **[TODO: Takeaway: which implicit constraints RL teaches the model to use proactively, and which it still fails on (the residual unit capabilities); connect back to the safety/robustness framing in Section 1. Reference the obstacle-avoidance probe (e.g. proactive BYPASS vs. collision) as a concrete instance.]**

5 Analysis and Discussion

[TODO: Failure-mode taxonomy via oracle diffing (Section 3.2); where RL helps (planning/profit) vs. where it lags (proactive safety); effect of action granularity on spatial reasoning; limitations (sim-to-real, single map, reward-hacking risks); broader impact (safety of embodied agents).]

6 Conclusion

[TODO: Restate: DELIVERYBENCH-RL makes long-horizon multimodal embodied RL feasible; RL baselines close part of the gap to scaffolded frontier VLMs but proactive implicit-constraint handling remains an open unit capability.]

A Environment Details

[TODO: Action/observation spaces, constraint definitions and costs, reward-term table, curriculum schedule, tool specifications, oracle-planner algorithm.]

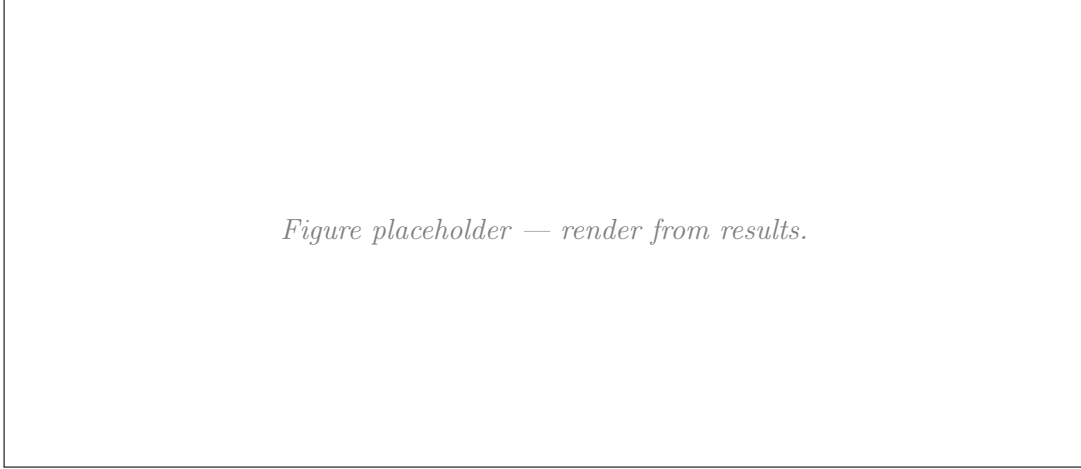


Figure 2: **Direct-QA accuracy vs. in-task proactive use, per constraint / subtask.** *What to plot:* grouped bars (or a radar) over constraints and subtasks—obstacle/safety, traffic-light, energy budget, deadline, packing, mode selection, . . .—with two bars per group: (a) accuracy when the rule is asked in *isolated QA*, and (b) the rate at which the model *proactively* applies the rule from within a live long-horizon state (extracted via the oracle unit tests of Section 3.2). The *gap* (a)–(b) quantifies the know-vs-act deficiency; show it for the base model and after RL.

B Training Details

[**TODO:** Hyperparameters per algorithm (PPO/GRPO/SUPO), context management, compute, wall-clock, seeds.]